Complete OS Viva Handbook & Technical Reference

Single-Core CPU Scheduling Simulator

> **Course:** Operating Systems
> **Topic:** CPU Scheduling, Process State Machine, Queueing Models, and Performance Metrics
> **Repository:** https://github.com/Aveeral/CPU-TASK-SCHEDULER

TABLE OF CONTENTS

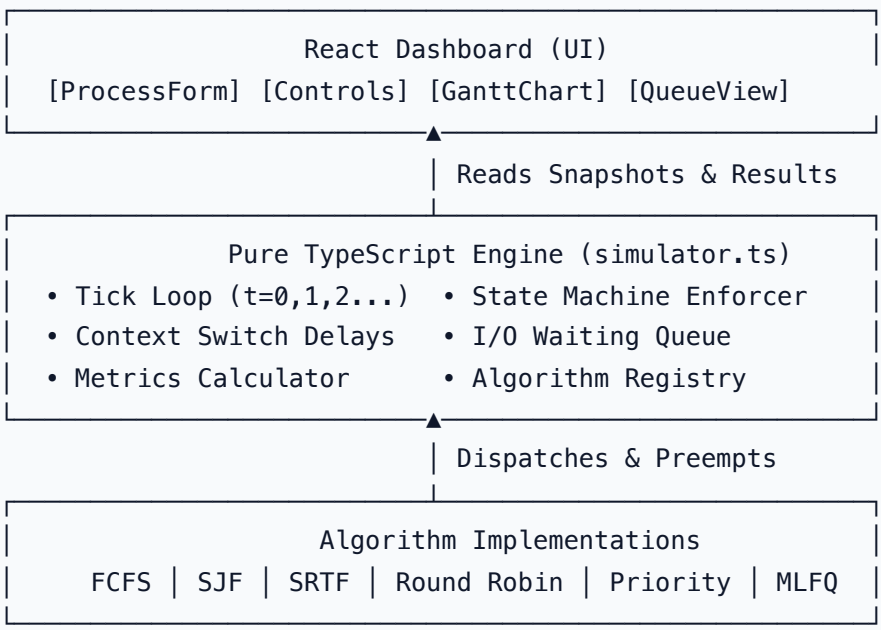
1. Architecture & System Design
2. Process Lifecycle & The State Machine
3. The 7-Step Simulation Engine Loop
4. In-Depth Scheduling Algorithms
5. Mathematical Formulas & Metric Calculations
6. Textbook Benchmarks & Test Cases
7. Top 15 Viva Questions & Model Answers
8. Demo Walkthrough Script for Presentation Day

1. Architecture & System Design

1.1 Decoupled Architecture (Engine vs. UI)

A core design requirement was strict decoupling between the simulation core and the user interface:

- `src/engine/` **(Pure TypeScript Core):**
  - **Zero UI / React imports.** Can be run in Node.js, CLI, or test runners without a browser DOM.
  - Implements Process Control Blocks (`PCB`), custom queues (`FIFOQueue` , `PriorityQueue` , `MultiLevelQueue`), scheduling algorithms, context-switch logic, and metrics calculation.
  - Time is discrete and modeled tick-by-tick ($t = 0, 1, 2, \ldots$).
- `src/components/` **(React Visualization Layer):**
  - Purely observational. Takes snapshots generated by the engine and renders the Gantt chart, Ready Queue, CPU state, and comparison charts.



2. Process Lifecycle & The State Machine

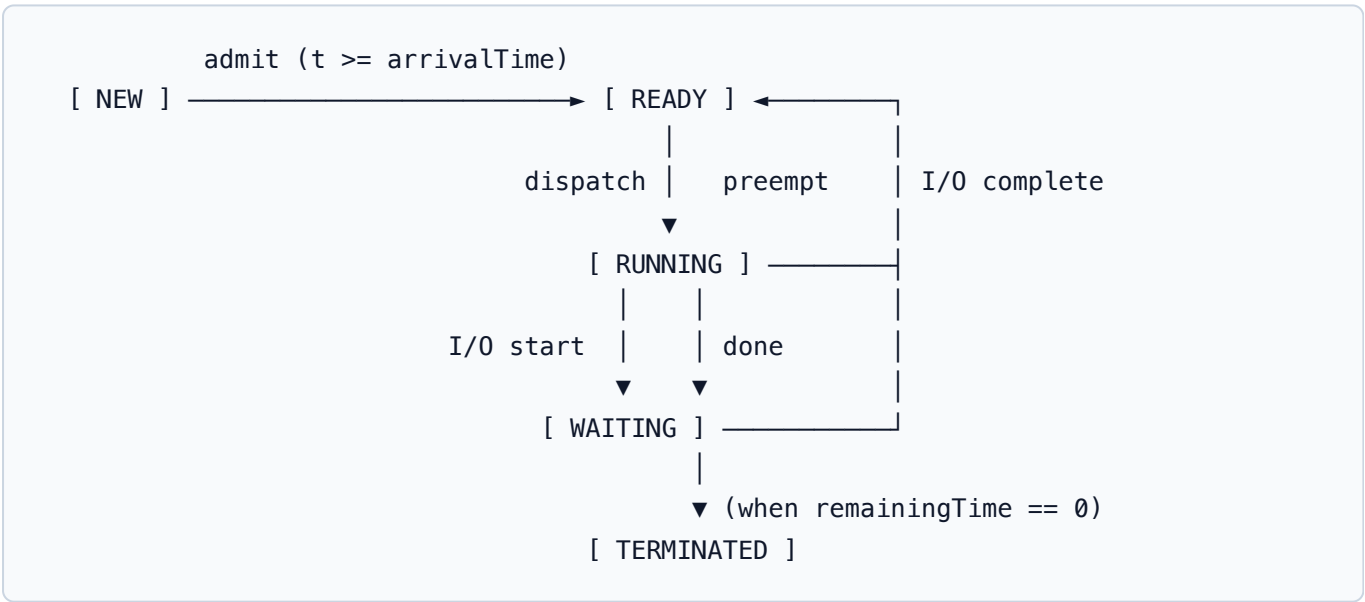
2.1 The 5 Legal Process States

The simulator implements the classical 5-state model defined in Silberschatz, Galvin & Gagne:

1. **`NEW`** : Process has been created with arrival time A and burst B, waiting for clock $t \geq A$.
2. **`READY`** : Process has arrived and is enqueued in the ready queue, waiting for CPU dispatch.
3. **`RUNNING`** : Process is currently allocated the single CPU core and executing instructions.
4. **`WAITING`** : Process is blocked waiting for an I/O burst to complete.
5. **`TERMINATED`** : Process has completed its total CPU burst ($remainingTime = 0$) and exited.

2.2 Transition Adjacency Map

Illegal transitions throw descriptive errors at runtime via `transition()` in `pcb.ts` :



Source State	Legal Destinations	Illegal Destinations (Throws Error)
`NEW`	`READY`	`RUNNING`, `WAITING`, `TERMINATED`
`READY`	`RUNNING`	`NEW`, `WAITING`, `TERMINATED`
`RUNNING`	`READY`, `WAITING`, `TERMINATED`	`NEW`
`WAITING`	`READY`	`NEW`, `RUNNING`, `TERMINATED`
`TERMINATED`	*(None — Terminal State)*	`NEW`, `READY`, `RUNNING`, `WAITING`

---

3. The 7-Step Simulation Engine Loop

At each discrete integer clock tick t, the simulation loop executes these 7 steps in exact order:

1. **Admit Arrivals (`NEW` → `READY`):** Any process with $p.state == NEW$ and $p.arrivalTime \leq t$ transitions to `READY` and calls `scheduler.onArrival(p, ctx)`.
2. **Update I/O Queue (`WAITING` → `READY`):** Processes in `waitingQueue` decrement `ioCountdown`. When `ioCountdown == 0`, the process transitions back to `READY` and calls `scheduler.onArrival(p, ctx)`.
3. **Context Switch Delay Handling:** If a context switch is in progress (`csCountdown > 0`), the CPU logs an **Idle** tick and advances t. No process executes productive instructions during this tick.
4. **Check Preemption:** If a process is currently running, the simulator calls `scheduler.shouldPreempt(running, ctx)`. If `true`:
   - `running` transitions to `READY` and is passed to `scheduler.onArrival(running, ctx)`.
   - `running` becomes `null`.
5. **Dispatch:** If the CPU is idle, `scheduler.pickNext(ctx)` selects the next job. If `config.contextSwitchCost > 0`, idle overhead ticks are inserted first.
6. **Execute 1 Tick:** The running process executes:
   - `running.remainingTime--`
   - `running.cpuTicksDone++`
   - `busyTicks++`
   - A snapshot is captured recording the exact system state for step-through playback.
7. **Handle Completion / I/O Burst:**
   - If `cpuTicksDone` matches an I/O burst trigger: process transitions to `WAITING` and moves to `waitingQueue`.
   - If `remainingTime == 0`: process transitions to `TERMINATED`, `completionTime` is recorded as $t + 1$.

---

4. In-Depth Scheduling Algorithms

4.1 FCFS (First-Come, First-Served)

- **Type:** Non-preemptive.
- **Queue Structure:** Standard FIFO queue.
- **Selection:** Dispatches the process that has been in the ready queue the longest.
- **Major Weakness:** The **Convoy Effect** — a long CPU-bound process monopolizes the CPU while short, interactive jobs pile up behind it, causing high average waiting time.

4.2 SJF (Shortest Job First - Non-Preemptive)

- **Type:** Non-preemptive.
- **Queue Structure:** Priority queue ordered by `burstTime`.
- **Selection:** Dispatches the process with the smallest total CPU burst. Once dispatched, it cannot be interrupted until it yields or terminates.
- **Optimality:** Provably minimizes average waiting time among all non-preemptive algorithms.

4.3 SRTF (Shortest Remaining Time First - Preemptive SJF)

- **Type:** Preemptive.
- **Queue Structure:** Priority queue ordered by `remainingTime`.
- **Preemption Rule:** \text{Preempt if: } \text{candidate.remainingTime} < \text{running.remainingTime}
- **Preemption Boundary:** If an arriving process has remaining time **equal** to the running process, it does **not** preempt (prevents useless context-switch thrashing).

4.4 Round Robin (RR)

- **Type:** Preemptive.
- **Queue Structure:** FIFO queue with a dynamic time quantum counter.
- **Quantum Countdown:** Tracks ticks consumed in the current time slice. When `ticksConsumed >= timeQuantum`:

- If `readyQueue` has waiting processes → Preempt running job, append it to the back of `readyQueue`.
- If `readyQueue` is empty → Renew quantum and continue without interruption (no artificial context-switch penalty).

- **Simultaneous Arrival vs. Quantum Expiry Rule:** At tick t, arrivals are admitted in **Step 1**, while preemption is evaluated in **Step 4**. Therefore: **Newly arriving jobs are enqueued BEFORE the preempted job returning to the queue.**

4.5 Priority Scheduling (Preemptive & Non-Preemptive)

- **Configurable Direction:**
  - `higherPriorityFirst = true` (Default OS convention): Lower integer = higher priority (0 is highest).
  - `higherPriorityFirst = false`: Larger integer = higher priority.
- **Preemptive Mode:** Preempts running job if an arrived job has strictly higher priority.
- **Starvation Solution (Aging):** Every `agingInterval` ticks, all jobs waiting in `readyQueue` have their priority incremented towards the highest priority.

4.6 MLFQ (Multi-Level Feedback Queue)

- **3 Priority Levels:**
  - **Level 0 (Top):** Round Robin with $q = 4$ (highest priority).
  - **Level 1 (Mid):** Round Robin with $q = 8$ (medium priority).
  - **Level 2 (Bottom):** FCFS (lowest priority, runs to completion).
- **Demotion Rule:** If a process exhausts its quantum at Level L, it is demoted to $\min(L + 1, 2)$.
- **Higher Queue Preemption:** A job in Level 0 strictly preempts a running job in Level 1 or 2.
- **Priority Boost:** Every `boostInterval` ticks (default 50), all jobs across all levels are promoted back to Level 0 to eliminate starvation.

---

5. Mathematical Formulas & Metric Calculations

Metric	Symbol	Mathematical Formula	Definition
Turnaround Time	TAT	TAT_i = C_i - A_i	Total elapsed time from arrival to completion.
Waiting Time	WT	WT_i = TAT_i - B_i	Total time spent waiting in the ready queue without CPU execution.
Response Time	RT	RT_i = S_i - A_i	Time from arrival until the process first acquires the CPU.
Average Waiting Time	$\bar{W}$	\bar{W} = \frac{1}{n} \sum_{i=1}^{n} WT_i	Mean wait time across all n jobs.
Average Turnaround	$\bar{T}$	\bar{T} = \frac{1}{n} \sum_{i=1}^{n} TAT_i	Mean turnaround time across all n jobs.
CPU Utilization		% U = \frac{\text{Busy Ticks}}{\text{Total Ticks}} \times 100	Percentage of time the CPU was executing productive process ticks.
Throughput	X	X = \frac{n}{TotalTime}	Completed processes per unit time.

6. Textbook Benchmarks & Test Cases

TC-1: FCFS Convoy Effect (from Silberschatz)

- **Inputs:** $P_1(A=0, B=24)$, $P_2(A=1, B=3)$, $P_3(A=2, B=3)$
- **Gantt:** $P_1[0-24] \rightarrow P_2[24-27] \rightarrow P_3[27-30]$
- **Metrics:**
  - $P_1 : TAT = 24, WT = 0, RT = 0$
  - $P_2 : TAT = 26, WT = 23, RT = 23$
  - $P_3 : TAT = 28, WT = 25, RT = 25$
  - $\bar{W} = 16.0$, $\bar{T} = 26.0$ (Convoy effect clearly demonstrated).

TC-3 & TC-4: SJF vs. SRTF Benchmark

- **Inputs:** $P_1(0, 7)$, $P_2(2, 4)$, $P_3(4, 1)$, $P_4(5, 4)$
- **SJF (Non-Preemptive):**
  - Gantt: $P_1[0-7] \rightarrow P_3[7-8] \rightarrow P_2[8-12] \rightarrow P_4[12-16]$
  - $\bar{W} = 4.0$, $\bar{T} = 8.0$
- **SRTF (Preemptive):**
  - Gantt: $P_1[0-2] \rightarrow P_2[2-4] \rightarrow P_3[4-5] \rightarrow P_2[5-7] \rightarrow P_4[7-11] \rightarrow P_1[11-16]$
  - $\bar{W} = 3.0$, $\bar{T} = 7.0$ (Preemption reduces average wait time from 4.0 to 3.0).

TC-5: Round Robin ($q = 4$)

- **Inputs:** $P_1(0, 10)$, $P_2(0, 6)$, $P_3(0, 2)$
- **Gantt:** $P_1[0-4] \rightarrow P_2[4-8] \rightarrow P_3[8-10] \rightarrow P_1[10-14] \rightarrow P_2[14-16] \rightarrow P_1[16-18]$
- **Metrics:**
  - $P_1 : TAT = 18, WT = 8, RT = 0$
  - $P_2 : TAT = 16, WT = 10, RT = 4$
  - $P_3 : TAT = 10, WT = 8, RT = 8$
  - $\bar{W} = 8.67$, $\bar{T} = 14.67$

7. Top 15 Viva Questions & Model Answers

Q1: Prove why SJF is mathematically optimal for minimizing average waiting time.

Answer: Let n processes run in order $P_1, P_2, \dots, P_n$ with burst times $b_1, b_2, \dots, b_n$. The total waiting time is: W_{total} = 0 + b_1 + (b_1 + b_2) + \dots + \sum_{j=1}^{n-1} b_j = \sum_{i=1}^{n} (n - i) b_i The coefficient $(n - i)$ represents how many subsequent jobs must wait for job i to finish. To minimize the sum of products $\sum (n - i) b_i$, the largest coefficients $(n - 1, n - 2, \dots)$ must multiply the smallest burst times b_i. By the **Exchange Argument**, swapping any out-of-order pair where $b_i > b_{i+1}$ strictly increases total waiting time. Thus, ordering by non-decreasing burst times yields the absolute minimum average waiting time.

Q2: If SJF is provably optimal, why don't general-purpose OSs use it?

Answer: Because in a real operating system, the kernel **cannot know the future length of a process's next CPU burst** before it executes. Operating systems address this by:

1. **Exponential Smoothing:** Predicting future bursts based on past history: \tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n
2. **Multi-Level Feedback Queues (MLFQ):** Inferring job characteristics dynamically: jobs that yield CPU quickly are assumed to be interactive (short bursts), while jobs that exhaust their time slices are demoted as CPU-bound.

Q3: What is the "Convoy Effect" in FCFS?

Answer: The convoy effect occurs when a long CPU-bound process holds the CPU while numerous short I/O-bound processes wait behind it in the ready queue. While the CPU-bound job runs, I/O devices sit completely idle. When the long process finishes and short jobs finally run, they quickly issue I/O, leaving the CPU idle. This causes poor hardware utilization and high average waiting times (e.g. TC-1 gives Avg WT = 16 in FCFS vs. 3 in SRTF).

Q4: Explain the trade-off in selecting the Round Robin time quantum (q).

Answer:

- **If q is too small ($q \to 0$):** Known as "processor sharing". System responsiveness is high, but the CPU spends an unacceptable fraction of time performing context switches instead of executing useful code.
- **If q is too large ($q \to \infty$):** Round Robin degrades into FCFS, reintroducing the convoy effect and high response times for interactive jobs.
- **Rule of Thumb:** Configure q such that **roughly 80% of CPU bursts are shorter than** q. Typically 10ms–100ms on modern hardware.

Q5: What is Starvation, and how does Aging prevent it?

Answer:

- **Starvation (Indefinite Postponement):** Occurs in priority scheduling or SJF when high-priority (or short) processes continuously enter the ready queue, preventing low-priority (or long) jobs from ever getting CPU time.
- **Aging:** Gradually increases the priority of waiting jobs proportional to their waiting time. Eventually, any starved job becomes the highest-priority job in the queue and is guaranteed CPU dispatch.

Q6: What physically happens during an OS context switch on real hardware?

Answer:

1. **Save CPU State:** General-purpose registers, program counter (PC), stack pointer (SP), and flags are written to the current process's PCB in kernel memory.
2. **Update Accounting:** Process state is changed (e.g. `RUNNING` $\to$ `READY`), CPU usage counters updated.
3. **Select Next Job:** Scheduler picks the next PCB from the ready queue.
4. **Memory Management Context Switch:** The page table base register ($CR3$ on x86, $TTBR0$ on ARM) is reloaded to point to the new process's address space.
5. **Cache Invalidation:** Translation Lookaside Buffer (TLB) entries must be flushed (unless tagged with Address Space IDs / ASIDs), and CPU caches ($L1/L2$) suffer temporary miss spikes ("cold cache").

6. **Restore CPU State:** The saved registers and PC of the new PCB are loaded into the hardware registers, and execution resumes.

Q7: In Round Robin, if process P_B arrives at the exact tick process P_A's quantum expires, which is queued first?

Answer: In our simulator and in standard OS theory (Silberschatz): **The newly arriving process (P_B) is enqueued BEFORE the preempted process (P_A).**

- Arrivals are evaluated at the beginning of the clock cycle (Step 1).
- Quantum preemption is evaluated in Step 4.
- Therefore, P_B enters the ready queue first, followed by P_A returning to the tail.

Q8: How does your simulator model context-switch overhead?

Answer: When `contextSwitchCost > 0` (e.g. 2 ticks), whenever `pickNext()` selects a new process, the process is held in a `pendingProcess` buffer while the CPU executes 2 explicit **Idle Overhead Ticks**. These ticks are logged as idle blocks on the Gantt chart and snapshots, allowing students to directly observe the throughput and CPU utilization penalties caused by excessive switching.

Q9: What are the 5 rules of MLFQ (Multi-Level Feedback Queue)?

Answer:

1. **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs and B does not.
2. **Rule 2:** If $\text{Priority}(A) == \text{Priority}(B)$, A and B run in Round Robin using the queue's quantum.
3. **Rule 3:** When a job enters the system, it is placed at the highest priority queue (Level 0).
4. **Rule 4:** Once a job uses up its time allotment at a given level, its priority is reduced (demoted by one queue).
5. **Rule 5:** After some time period S (Priority Boost), all jobs in the system are moved to the topmost queue to prevent starvation.

Q10: How does your scheduler handle deterministic tie-breaking?

Answer: Every queue comparator falls through to a deterministic two-level fallback:

1. **Primary Tie-Breaker: Arrival Time** (earliest arrived process runs first).
2. **Secondary Tie-Breaker: PID** (lowest process ID runs first). This eliminates nondeterministic race conditions across all algorithms.

Q11: What is the difference between Turnaround Time and Waiting Time?

Answer:

- **Turnaround Time ($TAT = C - A$):** Total lifespan of the process from arrival to termination (includes waiting time, CPU execution time, and any I/O time).
- **Waiting Time ($WT = TAT - Burst$):** The total duration spent idle inside the ready queue waiting for the CPU.

Q12: What is the difference between Turnaround Time and Response Time?

Answer:

- **Turnaround Time:** Time until the process finishes completely.
- **Response Time ($RT = StartTime - ArrivalTime$):** Time from when a process is submitted until it produces its **very first CPU response**. In interactive systems (e.g. desktop UI), Response Time is far more important than Turnaround Time.

Q13: In SRTF, does an arriving process preempt the running process if their remaining times are equal?

Answer: No. Preemption only occurs if the arrived job has **strictly smaller** remaining time ($\text{arrived.rem} < \text{running.rem}$). Preempting on equality would cause unnecessary context-switch overhead without reducing remaining execution time.

Q14: How are I/O bursts handled in your simulator?

Answer: Each process can declare an `ioBursts` schedule (e.g. after 2 CPU ticks, execute 3 ticks of I/O). When `cpuTicksDone == io.afterCpu`:

1. Process transitions from `RUNNING` $\rightarrow$ `WAITING`.
2. Process is enqueued in `ctx.waitingQueue` with an `ioCountdown`.

3. The CPU is freed, allowing other ready jobs to execute.
4. Each tick, `ioCountdown` decrements. When it hits 0, the process transitions `WAITING` → `READY`.

Q15: Why is your simulator engine written in pure TypeScript with no React dependencies?

Answer:

1. **Separation of Concerns:** OS kernel scheduling logic is independent of presentation technology.
2. **Testability:** The engine can be tested with Vitest without mocking browser DOM elements, resulting in fast execution (210 tests run in ~1 second).
3. **Portability:** The engine can be compiled to Node.js CLI tools, Web Workers, or embedded systems without touching UI code.

---

8. Demo Walkthrough Script for Presentation Day

When presenting to your professor/evaluator, follow this structured 2-minute flow:

1. **Introduction (15 seconds):**

   > *"Good morning. I developed a Single-Core CPU Scheduling Simulator with a decoupled pure TypeScript simulation engine and an interactive React visualizer. It models 7 scheduling policies, discrete tick execution, context-switch overhead, and I/O bursts."*

2. **Demonstrate the Convoy Effect (30 seconds):**
   - Click **Load Sample Data** → Pick **TC-1: FCFS Basic (3 processes)**.
   - Click ⚡ **Run All**.
   - Show the Gantt chart:

     > *"Notice process P1 with burst 24 holds the CPU from t=0 to t=24. Short processes P2 and P3 wait 23 and 25 ticks respectively, leading to a high Average Waiting Time of 16.0 ticks."*

3. **Demonstrate SJF/SRTF Optimality (30 seconds):**
   - Click **Algorithm** dropdown → Select **SRTF**.
   - Click ⚡ **Run All**.
   - Show how Average Waiting Time drops:

     > *"With the same workload, SRTF immediately schedules P2 and P3 first, dropping the Average Waiting Time from 16.0 down to 3.0 ticks. This demonstrates the mathematical optimality of shortest remaining time scheduling."*

4. **Demonstrate Step-Through & Ready Queue (30 seconds):**
   - Click ↺ **Reset**.

- Click **Step Forward ⏭** a few times:

> *"As we step tick-by-tick, notice the Active CPU Core progress bar updating, the live Ready Queue dynamically reordering according to priority, and the Gantt chart recording each execution segment."*

5. **Show Multi-Algorithm Benchmark (15 seconds):**
   - Click the 📊 **Compare All Algorithms** tab.
   - Point to the comparative bar chart:

> *"The comparison view runs all 7 algorithms simultaneously on the identical workload, plotting Average Waiting Time and Turnaround Time side-by-side and highlighting the optimal policy."*